ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO BÀI TẬP LỚN 1

ĐỀ TÀI: DISTRIBUTED AND STREAMING-BASED CLIENT-SERVER COLLABORATION APPLICATION

Giảng viên hướng dẫn:	Bùi Xuân Giang
Lớp:	L01
Nhóm thực hiện:	02

TP. Hồ Chí Minh, tháng 11/2026

DANH SÁCH THÀNH VIÊN NHÓM

STT	Thành viên nhóm	MSSV	Lớp	Đóng góp
1	Ngô Tuấn Khôi	2411697	L01	100%
2	Đặng Phúc Khang	2411421	L01	100%
3	Huỳnh Tiến Dũng	2410587	L01	100%

Mục lục

1 Tổng quan lý thuyết **3**
1.1 Mô hình OSI & Bộ Giao thức TCP/IP . . . 3
1.2 So sánh: TCP Stream vs. UDP Datagram . . . 3
1.3 Head-of-Line Blocking & Ngưỡng MTU . . . 3

2 Kiến trúc Hệ thống (System Architecture) **4**
2.1 Nguyên lý Phân tách Mặt phẳng . . . 4
2.2 Sơ đồ Kiến trúc Tổng thể Hệ thống . . . 4
2.3 Kiến trúc Web Server HTTP 1.1 Thuần trên Socket . . . 4

3 Đặc tả Giao thức Tầng Ứng dụng **5**
3.1 Cấu trúc Header TCP Nhị phân 20 Bytes (Kênh Signaling) . . . 5
3.2 Cấu trúc Header UDP Nhị phân 24 Bytes (Kênh Data Streaming) . . . 5
3.3 Danh mục Loại Thông điệp (Message Types Enum) . . . 5
3.4 Đối soát Toàn vẹn: Băm SHA-256 vs. Checksum CRC-32 . . . 6

4 Cơ chế Truyền Tin cậy & Toán học ARQ **7**
4.1 So sánh các Thuật toán Truyền Tin cậy: Stop-and-Wait, Go-Back-N, Selective Repeat . . . 7
4.2 Hiệu năng Truyền dẫn . . . 7
4.3 Sequence Number Wrap-Around . . . 7

5 Sơ đồ Tuần tự Hoạt động (Sequence Diagrams) **8**
5.1 Sơ đồ 1: Bắt tay TCP & Đăng nhập Xác thực . . . 8
5.2 Sơ đồ 2: Nhắn tin Thời gian thực Mã hóa E2EE . . . 9
5.3 Sơ đồ 3: Truyền Tệp tin Tin cậy qua UDP (Selective Repeat ARQ) . . . 10
5.4 Sơ đồ 4: Giữ nhịp tim (Heartbeat 10s) & Watchdog Tự dọn dẹp (15s) . . . 11

6 Khó khăn & Giải pháp (Challenges & Solutions) **12**
6.1 Dính gói & Xé gói của TCP . . . 12
6.2 Đồng thời, Xung đột Dữ liệu & Chặn Deadlock . . . 12
6.3 Tái sử dụng Cổng & TIME_WAIT . . . 12
6.4 Phục hồi Kết nối khi Client Sập nguồn Đột ngột . . . 12
6.5 Kết luận . . . 13

1 Tổng quan lý thuyết

1.1 Mô hình OSI & Bộ Giao thức TCP/IP

Hệ thống được xây dựng tuân thủ nghiêm ngặt mô hình phân tầng của mạng máy tính:

- **Tầng Ứng dụng (Application Layer):** Thiết kế giao thức nhị phân tùy biến (*Custom Binary Framing Protocol*) với Header nhị phân cố định, phân tách rõ ràng giữa siêu dữ liệu (Metadata) và tải trọng (Payload).
- **Tầng Giao vận (Transport Layer):** Khai thác đồng thời hai giao thức: **TCP** (hướng kết nối, bảo đảm tin cậy) cho kênh điều khiển và **UDP** (phi kết nối, hướng datagram tốc độ cao) cho kênh truyền dữ liệu khối lượng lớn.
- **Tầng Mạng (Network Layer - IP):** Đảm bảo kích thước phân đoạn ứng dụng không vượt quá ngưỡng **MTU (Maximum Transmission Unit) = 1500 bytes** của Ethernet nhằm triệt tiêu hoàn toàn rủi ro phân mảnh tầng mạng (*IP Fragmentation*).

1.2 So sánh: TCP Stream vs. UDP Datagram

Trong thiết kế ứng dụng cộng tác thời gian thực, việc lựa chọn giao thức tầng giao vận quyết định trực tiếp đến độ trễ và thông lượng:

Đặc tính Kỹ thuật	TCP	UDP
Mô hình truyền	Hướng kết nối (Connection-oriented), Byte-stream liên tục.	Phi kết nối (Connectionless), Datagram rời rạc độc lập.
Bắt tay (Handshake)	3-way handshake (SYN → SYN-ACK → ACK) gây trễ $1 \times$ RTT.	Không bắt tay, truyền tức thì ($0 \times$ RTT).
Độ tin cậy	Đảm bảo 100%: Tự động truyền lại, sửa lỗi, xếp đúng thứ tự.	Best-effort: Có thể mất, trùng lặp hoặc đảo lộn thứ tự gói tin.
Kiểm soát lưu lượng	Flow Control (Sliding Window), Congestion Control (AIMD/CUBIC).	Không có kiểm soát ở kernel; tầng ứng dụng tự điều tiết.
Ứng dụng trong dự án	**Mặt phẳng Báo hiệu (Signaling Plane):** Bắt tay, Đăng nhập, Chat E2EE, Heartbeat.	**Mặt phẳng Dữ liệu (Data Streaming Plane):** Truyền file khối lượng lớn với Selective Repeat ARQ.

Bảng 1: So sánh giữa TCP và UDP.

1.3 Head-of-Line Blocking & Ngưỡng MTU

1. **Head-of-Line (HoL) Blocking trong TCP:** Trên một kết nối TCP đơn, kernel hệ điều hành bắt buộc phải bàn giao dữ liệu cho ứng dụng theo đúng thứ tự tuần tự tuyệt đối. Nếu truyền một tệp tin dung lượng lớn chung với tin nhắn chat: khi chỉ một gói tin của tệp bị mất trên đường truyền, toàn bộ các tin nhắn chat và gói tin giữ nhịp tim (`HEARTBEAT_PING`) đến sau đều bị giữ lại trong hàng đợi buffer . Hậu quả là xuất hiện *Latency Spike* (độ trễ nhảy vọt), giao diện đóng băng và Watchdog có thể ngắt kết nối nhầm do nghi ngờ Client bị treo.
2. **Ngưỡng MTU & Phân mảnh IP:** Kích thước truyền tối đa của mạng Ethernet tiêu chuẩn là MTU = 1500 bytes. Trừ đi IPv4 Header (20B) và UDP Header (8B), tải trọng tối đa trước khi bị Router băm nhỏ là $1500 - 28 = 1472$ bytes. Nếu vượt quá, Router buộc phải thực hiện *IP Fragmentation*. Khi một mảnh bị mất, toàn bộ Datagram bị hủy mà không có cơ chế truyền lại ở tầng IP. Do đó, phải khống chế kích thước chunk ở mức **1200** bytes $<$ 1472 bytes, đảm bảo an toàn tuyệt đối.

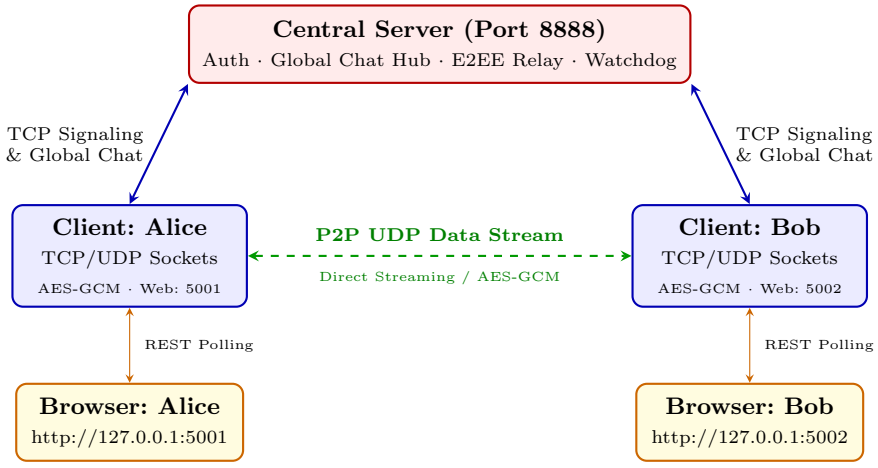
2 Kiến trúc Hệ thống (System Architecture)

2.1 Nguyên lý Phân tách Mặt phẳng

Lấy tham khảo từ kiến trúc của Google QUIC và WebRTC, hệ thống phân tách hoàn toàn thành hai mặt phẳng độc lập:

- **Mặt phẳng Báo hiệu (TCP Signaling Plane – Port 8888):** Duy trì kết nối TCP bền vững, đảm bảo tính toàn vẹn 100% cho các tác vụ quản lý trạng thái, đồng bộ danh bạ online, chuyển tiếp tin nhắn mã hóa đầu-cuối (E2EE) và gửi gói tin giữ nhịp tim.
- **Mặt phẳng Dữ liệu (UDP Data Streaming Plane – Port 8889 / Dynamic P2P):** Dữ liệu tệp tin được truyền trực tiếp giữa các Client (Peer-to-Peer) hoặc qua Server Relay bằng UDP datagrams, loại bỏ hoàn toàn ảnh hưởng của việc truyền dữ liệu nặng lên kênh điều khiển.

2.2 Sơ đồ Kiến trúc Tổng thể Hệ thống



Hình 1: Kiến trúc hệ thống (System Architecture).

2.3 Kiến trúc Web Server HTTP 1.1 Thuần trên Socket

- Lớp `RawSocketHTTPServer` (`client/web_server.py`) lắng nghe trực tiếp trên cổng TCP cục bộ.
- Tự phân tích chuỗi byte HTTP thô: Tách dòng Request Line (`METHOD /path HTTP/1.1`), phân giải từ điển Header (`Key: Value`) ngăn cách bởi ký tự `\r\n\r\n`, và đọc đúng số byte Body dựa vào trường `Content-Length`.
- Thay vì dùng WebSockets, Web Client sử dụng kỹ thuật **Periodic REST Polling** (tần số 1000ms qua lệnh `fetch('/api/state')`) để cập nhật danh bạ người dùng, tin nhắn chat mới và thanh tiến trình truyền file thời gian thực.

3 Đặc tả Giao thức Tầng Ứng dụng

3.1 Cấu trúc Header TCP Nhị phân 20 Bytes (Kênh Signaling)

Giao thức định nghĩa cấu trúc Header nhị phân cố định đúng **20 bytes**, định dạng theo chuẩn mạng Big-Endian (`!HBBB3sIII`):

Trường (Field)	Kích thước	Định dạng Struct	Ý nghĩa
`Magic Number`	2 bytes	`!H` (unsigned short)	Giá trị cố định `0x4353` (ASCII 'CS' – Collaboration Streaming) dùng để kiểm chứng gói tin hợp lệ và đồng bộ biên khung.
`Version`	1 byte	`B` (unsigned char)	Phiên bản giao thức (Version = 1).
`MsgType`	1 byte	`B` (unsigned char)	Định danh phân loại thông điệp (1 đến 30).
`Flags`	1 byte	`B` (unsigned char)	Cờ điều khiển: `0x00` (Plaintext), `0x01` (Encrypted với AES-256-GCM).
`Reserved`	3 bytes	`3s` (bytes string)	Padding căn lề 4-byte (**Data Alignment**) trong bộ nhớ CPU 32/64-bit (`b"\x00\x00\x00"`).
`Sequence`	4 bytes	`I` (unsigned int)	Số thứ tự gói tin tăng đơn điệu phục vụ chống trùng lặp.
`Length`	4 bytes	`I` (unsigned int)	Độ dài chính xác của phần tải trọng dữ liệu (Payload) theo sau ($0 \leq L \leq 2^{32} - 1$).
`Checksum`	4 bytes	`I` (unsigned int)	Mã kiểm tra toàn vẹn **CRC32** của Payload (`zlib.crc32(payload) & 0xFFFFFFFF`).

Bảng 2: Đặc tả chi tiết từng bit của Header TCP 20 Bytes (`protocol/packet.py`).

3.2 Cấu trúc Header UDP Nhị phân 24 Bytes (Kênh Data Streaming)

Kênh truyền tệp tin sử dụng Header nhị phân cố định đúng **24 bytes** (`!HBBIIIH2sI`):

Trường (Field)	Kích thước	Định dạng Struct	Ý nghĩa
`Magic Number`	2 bytes	`!H` (unsigned short)	Giá trị cố định `0x5546` (ASCII 'UF' – UDP Framing).
`Type`	1 byte	`B` (unsigned char)	Phân loại segment: 1 (`DATA`), 2 (`ACK`), 3 (`FIN`).
`Flags`	1 byte	`B` (unsigned char)	Cờ mã hóa chunk payload (`0x01` = Encrypted).
`StreamID`	4 bytes	`I` (unsigned int)	Định danh luồng truyền độc nhất cho mỗi phiên gửi tệp.
`ChunkIdx`	4 bytes	`I` (unsigned int)	Chỉ số thứ tự của chunk hiện tại ($0, 1, \ldots,$ Total-1).
`TotalChunks`	4 bytes	`I` (unsigned int)	Tổng số lượng chunk tạo nên tệp tin hoàn chỉnh.
`DataLen`	2 bytes	`H` (unsigned short)	Kích thước thực tế của dữ liệu trong chunk (tối đa 1200 bytes).
`Reserved`	2 bytes	`2s` (bytes string)	Trường đệm căn lề 4-byte (`b"\x00\x00"`).
`CRC32`	4 bytes	`I` (unsigned int)	Mã kiểm tra lỗi nhị phân độc lập cho từng chunk payload.

Bảng 3: Đặc tả chi tiết cấu trúc Header UDP 24 Bytes (`protocol/udp_stream.py`).

3.3 Danh mục Loại Thông điệp (Message Types Enum)

Nhóm nghiệp vụ	Mã Type	Tên Enum	Mô tả chức năng & Dữ liệu đính kèm
Bắt tay & Xác thực	1	`HANDSHAKE_REQ`	Khởi tạo kết nối, Client gửi chuỗi nhận diện.
	2	`HANDSHAKE_ACK`	Server chấp thuận, gửi kèm cổng tiếp nhận UDP 8889.
	3	`AUTH_LOGIN`	Client gửi yêu cầu đăng nhập: `{"username", "udp_port"}`.
	4	`AUTH_SUCCESS`	Đăng nhập thành công, Server gửi danh sách online peers.
	5	`AUTH_FAILURE`	Từ chối đăng nhập (tên rỗng hoặc đã có người chiếm dụng).
	6	`DISCONNECT`	Client chủ động ngắt kết nối an toàn (Graceful exit).
Trạng thái & Nhịp tim	12	`USER_JOINED`	Server broadcast người mới gia nhập cho toàn mạng.
	13	`USER_LEFT`	Server broadcast người rời phòng (kèm lý do time-out/quit).
	14	`HEARTBEAT_PING`	Client gửi nhịp tim định kỳ mỗi 10 giây.
	15	`HEARTBEAT_PONG`	Server xác nhận duy trì phiên kết nối.
Nhắn tin Thời gian thực	20	`MSG_BROADCAST`	Tin nhắn phòng chung (Zero-Knowledge E2EE Relay).
	21	`MSG_UNICAST`	Tin nhắn riêng tư trực tiếp đích danh giữa 2 Client.
	22	`MSG_DELIVERY_ACK`	Server xác nhận tin nhắn đã chuyển tiếp thành công.
Báo hiệu Truyền tệp	30	`FILE_ANNOUNCE`	Báo hiệu trước thông tin file: tên, kích thước, tổng chunk và mã SHA-256.

Bảng 4: Bảng mã định danh thông điệp trong NexusStream Protocol.

3.4 Đối soát Toàn vẹn: Băm SHA-256 vs. Checksum CRC-32

Hệ thống kết hợp kiểm tra toàn vẹn hai cấp độ:

Đặc tính kỹ thuật	CRC-32 (Cyclic Redundancy Check)	SHA-256 (Secure Hash Algorithm)
Kích thước ngõ ra	32 bits (4 bytes).	256 bits (32 bytes / 64 ký tự hex).
Mục đích chính	Phát hiện lỗi truyền dẫn ngẫu nhiên (nhiễu bit phần cứng, rớt gói).	Kiểm tra tính toàn vẹn mật mã học (chống giả mạo, kháng va chạm).
Tốc độ tính toán	Cực nhanh (phép chia đa thức nhị phân bằng phần cứng/bảng tra).	Chậm hơn đáng kể (nhiều vòng băm mật mã học phức tạp).
Ứng dụng trong dự án	Đặt trong Header của từng gói tin TCP/UDP để kiểm tra hỏng hóc tức thì.	Tính trên toàn bộ tệp tin trước khi gửi và sau khi ráp file hoàn chỉnh.

Bảng 5: Đối chiếu kỹ thuật giữa CRC-32 và SHA-256 trong hệ thống.

4 Cơ chế Truyền Tin cậy & Toán học ARQ

4.1 So sánh các Thuật toán Truyền Tin cậy: Stop-and-Wait, Go-Back-N, Selective Repeat

Để đảm bảo truyền file không mất mát dữ liệu trên nền giao thức UDP không tin cậy, ta xem xét 3 cơ chế ARQ kinh điển:

1. **Stop-and-Wait:** Bên gửi phát 1 gói, dừng lại chờ ACK từ bên nhận rồi mới phát gói tiếp theo. Hiệu suất đường truyền cực thấp: $U_{\text{SW}} \approx \frac{L/R}{\text{RTT}+L/R} \approx 0.05$, đường truyền hầu như rảnh rỗi.

2. **Go-Back-N (GBN):** Cho phép phát liên tục đến N gói (Pipelining) với cơ chế ACK tích lũy (*Cumulative ACK*). Nhược điểm nghiêm trọng: Nếu mất gói thứ i, toàn bộ các gói từ i đến $i+N-1$ đến sau đều bị phía nhận vứt bỏ; phía gửi buộc phải truyền lại tất cả các gói đó, gây lãng phí băng thông nghiêm trọng trên đường truyền mất gói.

3. **Selective Repeat (SR):** Cho phép phát liên tục đến N gói trong cửa sổ trượt. Phía nhận duy trì bộ đệm (`received_chunks`) để lưu tạm các gói đến trước, và gửi lại **Selective ACK** cho từng gói nhận thành công. Phía gửi duy trì bộ đếm thời gian riêng và **chỉ truyền lại duy nhất gói bị mất/timeout**. Đây là thuật toán được chọn hiện thực trong NexusStream!

4.2 Hiệu năng Truyền dẫn

Gọi L là kích thước gói tin (bits), R là băng thông (bps), RTT là độ trễ khứ hồi:

$$\text{Hiệu suất kênh Stop-and-Wait:} \quad U_{\text{SW}} = \frac{L/R}{\text{RTT} + L/R}$$

$$\text{Hiệu suất kênh Selective Repeat } (N = 8): \quad U_{\text{SR}} = \min\left(1, \frac{N \times L/R}{\text{RTT} + L/R}\right)$$

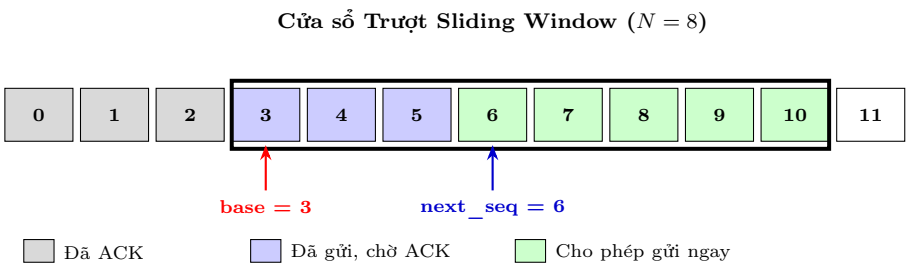
Với kích thước cửa sổ trượt $N = 8$, đường truyền được lấp đầy thông qua kỹ thuật Pipelining, giúp thông lượng thực tế tăng gấp gần 8 lần so với mô hình Stop-and-Wait.

4.3 Sequence Number Wrap-Around

Trong giao thức Selective Repeat, để phía nhận không bao giờ nhầm lẫn giữa một gói tin mới và một gói tin truyền lại sau khi số thứ tự quay vòng (Wrap-around), kích thước cửa sổ gửi W_{send} và cửa sổ nhận W_{recv} phải thỏa mãn điều kiện toán học:

$$W_{\text{send}} + W_{\text{recv}} \leq 2^k \implies N \leq 2^{k-1}$$

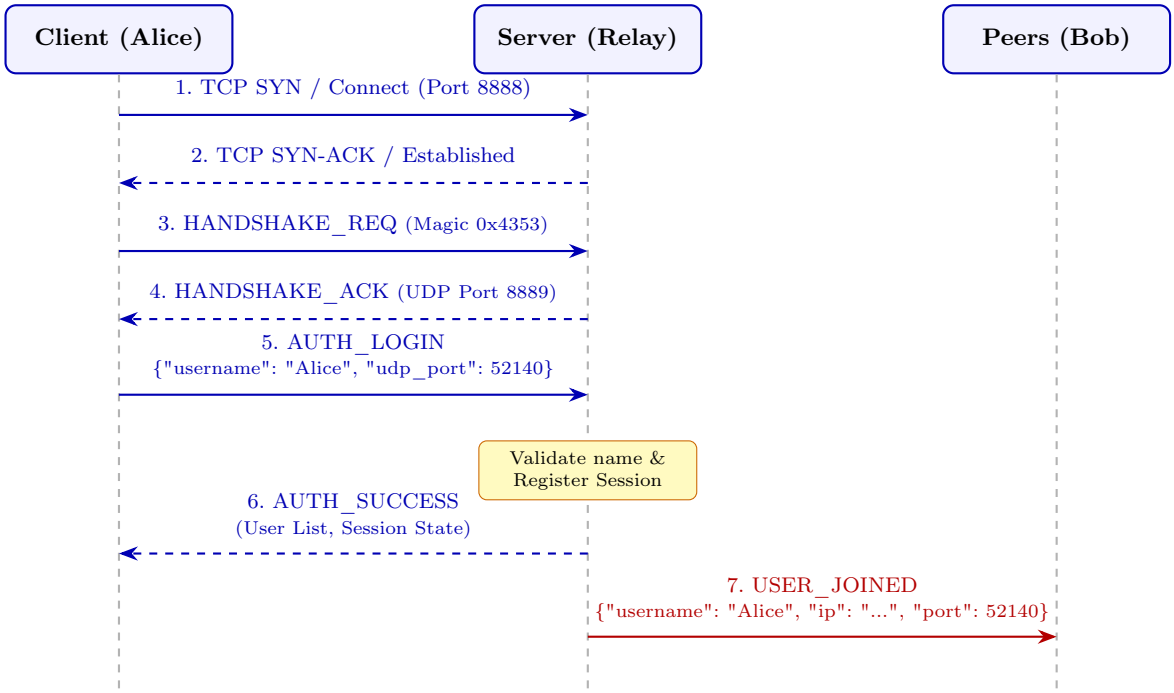
Trong đó k là số bit biểu diễn trường số thứ tự. Trong NexusStream, trường `ChunkIdx` có kích thước 32-bit ($k = 32 \implies 2^{32} \approx 4.29 \times 10^9$). Do kích thước cửa sổ $N = 8 \ll 2^{31}$, hệ thống **hoàn toàn miễn nhiễm** với hiện tượng trùng lặp số thứ tự.



Hình 2: Mô hình quản lý Cửa sổ Trượt (Sliding Window $N = 8$) trong `protocol/udp_stream.py`.

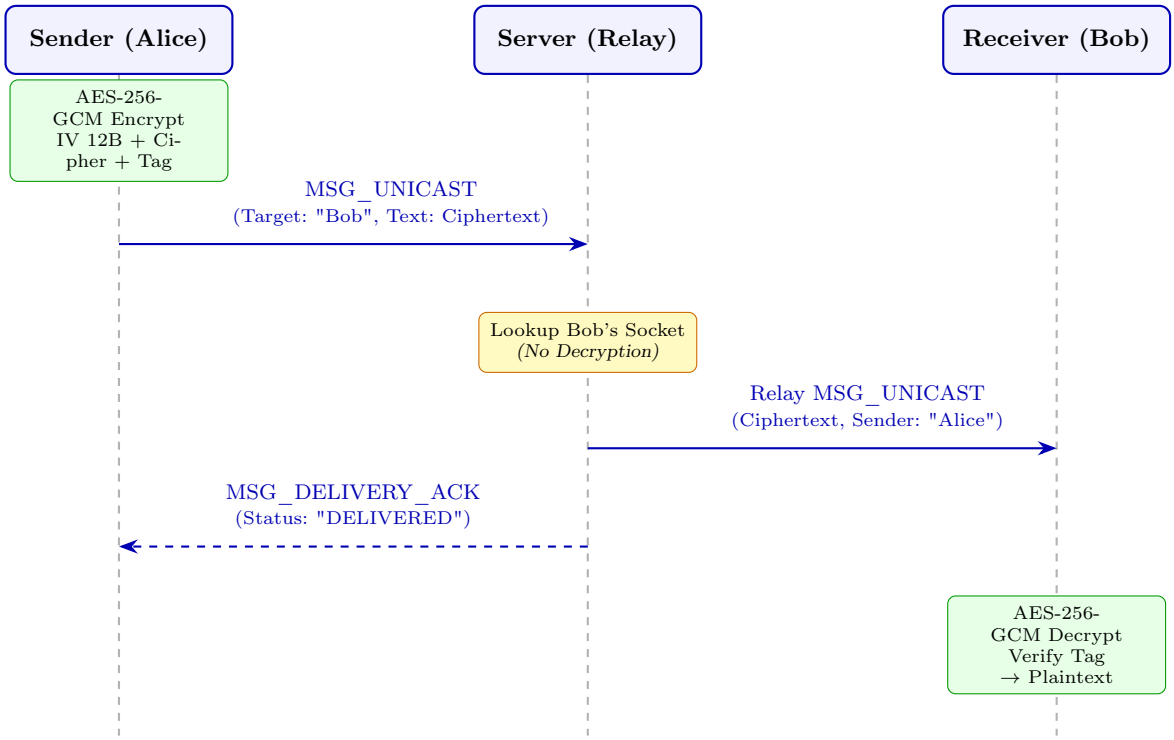
5 Sơ đồ Tuần tự Hoạt động (Sequence Diagrams)

5.1 Sơ đồ 1: Bắt tay TCP & Đăng nhập Xác thực



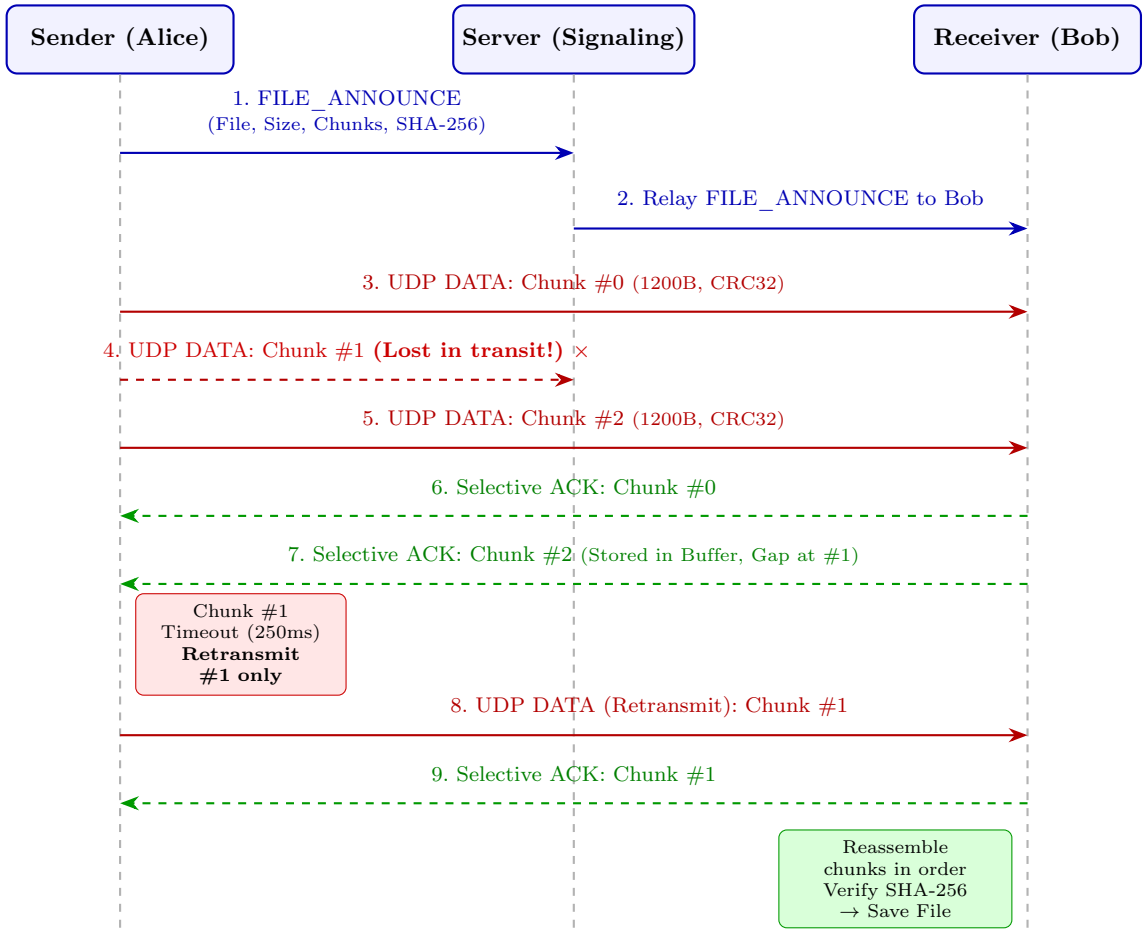
Hình 3: Quy trình bắt tay 2 chiều TCP và Đăng nhập xác thực.

5.2 Sơ đồ 2: Nhắn tin Thời gian thực Mã hóa E2EE



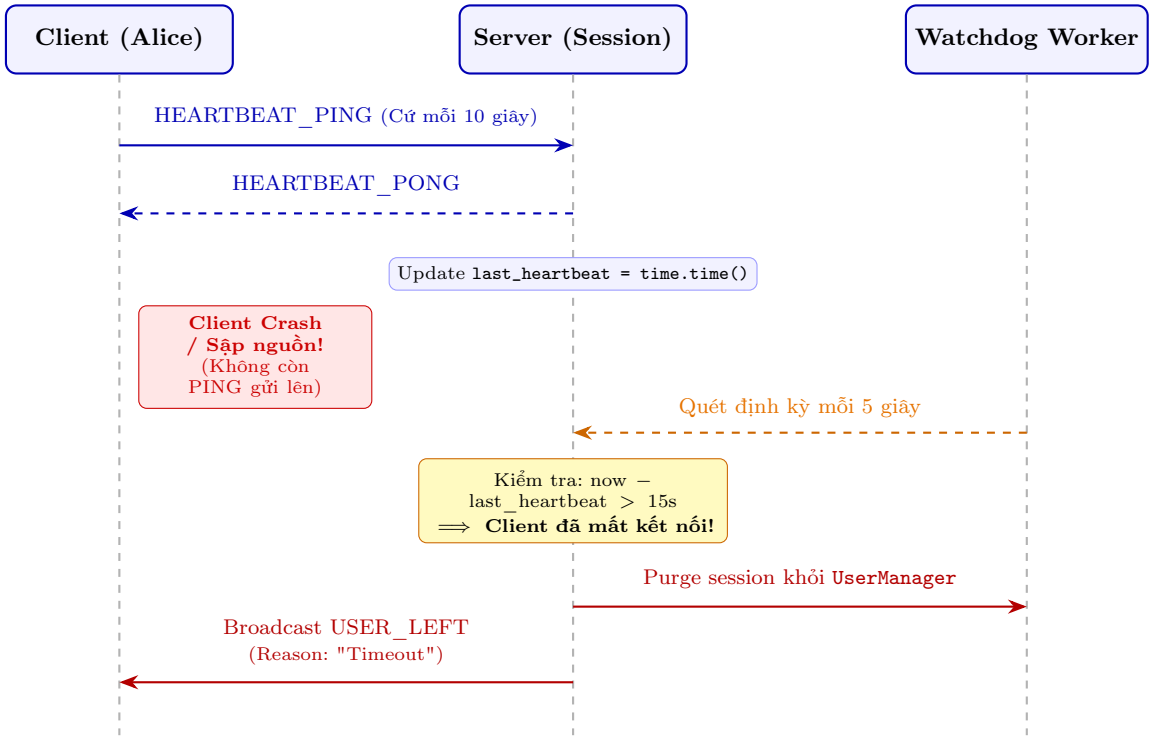
Hình 4: Truyền tin nhắn Unicast theo nguyên lý Zero-Knowledge E2EE Relay.

5.3 Sơ đồ 3: Truyền Tệp tin Tin cậy qua UDP (Selective Repeat ARQ)



Hình 5: Vòng đời truyền tệp tin tin cậy qua UDP với kỹ thuật khôi phục Selective Retransmission.

5.4 Sơ đồ 4: Giữ nhịp tim (Heartbeat 10s) & Watchdog Tự dọn dẹp (15s)



Hình 6: Cơ chế duy trì nhịp tim và Watchdog Worker tự phục hồi tài nguyên khi Client sập nguồn.

6 Khó khăn & Giải pháp (Challenges & Solutions)

6.1 Dính gói & Xé gói của TCP

- **Vấn đề:** TCP là giao thức hướng dòng byte (Byte-Stream Protocol), không duy trì ranh giới thông điệp. Lời gọi `sock.recv(buffer_size)` chỉ đọc dữ liệu hiện có trong bộ đệm hệ điều hành. Nếu mạng tắc nghẽn, gói tin bị xé nhỏ, khiến ứng dụng chỉ nhận được một mẩu header; ngược lại, nếu bên gửi phát nhanh, nhiều gói tin bị dính chùm trong bộ đệm.
- **Giải pháp:** Thiết kế Header cố định 20 bytes chứa trường `Length` và xây dựng hàm `recv_exact(sock, num_bytes)`:

```
def recv_exact(sock, num_bytes):
    """Doc chinh xac num_bytes tu TCP stream de chong dinh/xe goi tuyet doi."""
    buffer = bytearray()
    while len(buffer) < num_bytes:
        chunk = sock.recv(num_bytes - len(buffer))
        if not chunk:
            raise ConnectionResetError("Remote socket connection closed
    unexpectedly")
        buffer.extend(chunk)
    return bytes(buffer)
```

Hàm giải mã `recv_packet()` luôn đọc đúng 20 bytes đầu tiên để trích xuất độ dài `length`, sau đó tiếp tục đọc chính xác `length` bytes để gom đủ payload trước khi unpack.

6.2 Đồng thời, Xung đột Dữ liệu & Chặn Deadlock

- **Vấn đề:** Máy chủ phục vụ hàng chục Client kết nối và ngắt kết nối đồng thời. Việc ghi nhận và cập nhật danh bạ người dùng trực tuyến (`_users` dict) rất dễ xảy ra hiện tượng xung đột. Hơn nữa, nếu các hàm nội bộ lồng nhau cùng yêu cầu chiếm dụng khóa `threading.Lock()` thông thường, luồng sẽ tự khóa chính mình và gây ra hiện tượng **Self-Deadlock**.
- **Giải pháp:** Sử dụng khóa đệ quy `threading.RLock()` trong lớp `UserManager` và `CollaborationClient`. `RLock` cho phép luồng hiện tại được tái chiếm khóa nhiều lần mà không bị treo máy. Đồng thời, cấu trúc `with session.lock:` được bọc quanh thao tác `send_packet()` của từng socket để tránh xung đột byte khi nhiều luồng cùng gửi tin nhắn tới một máy khách.

6.3 Tái sử dụng Cổng & TIME_WAIT

- **Vấn đề:** Khi máy chủ bị ngắt và khởi động lại ngay lập tức, socket TCP rơi vào trạng thái `TIME_WAIT` của hệ điều hành nhằm đảm bảo các gói tin cũ trên mạng tiêu biến hết. Nếu không có cấu hình đặc biệt, lệnh `bind()` sẽ lập tức ném lỗi `OSError: [Errno 98] Address already in use` (hoặc `WinError 10048`).
- **Giải pháp:** Thiết lập tùy chọn socket trước khi bind:

```
self.tcp_sock.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
```

Cờ này cho phép kernel tái sử dụng ngay lập tức cặp địa chỉ IP:Port, giúp Server có thể phục hồi tức thì sau sự cố mà không cần thời gian chờ.

6.4 Phục hồi Kết nối khi Client Sập nguồn Đột ngột

- **Vấn đề:** Khi client bị sập nguồn đột ngột hoặc mất mạng, socket TCP rơi vào trạng thái *Half-Open Connection*, Server không nhận được gói tin FIN hay RST và có nguy cơ rò rỉ tài nguyên socket vĩnh viễn.
- **Giải pháp:** Kết hợp giữa ngoại lệ socket và tiến trình Watchdog:
  - Bắt toàn bộ các ngoại lệ cấp thấp: `ConnectionResetError`, `BrokenPipeError`, `socket.timeout`.
  - Luồng `Watchdog-Worker` chạy ngầm định kỳ 5 giây, tự động thanh trừng các session có now − last_heartbeat > 15.0s, giải phóng bộ nhớ và gửi gói `USER_LEFT` cập nhật danh bạ toàn mạng.

6.5 Kết luận

Báo cáo kỹ thuật và mã nguồn BTL đã hiện thực yêu cầu môn học. Bằng cách vận dụng socket nhị phân mức thấp kết hợp với các nguyên lý thiết kế mạng tiên tiến (Dual-Plane Architecture, Selective Repeat ARQ, AES-256-GCM AEAD và Watchdog Resilience), hệ thống đã cơ bản đúng về mặt lý thuyết và độ tin cậy trong triển khai.